

README of ZorkUL (Roguelike-RPG)

Maker: Haolong Qi, Xiaozhou Liu

I. 代码内新增关键类

• ZorkUL

- Data Members:

1. parser: Parser //指令解析
2. currentRoom: Room* //当前房间
3. player: Character* //角色指针
4. itemList: vector< Item* > //道具列表 -> 体现多态
5. roomList: vector< Room* > //房间列表
6. monsterList: vector< Monster* > //怪物列表

• character

- Data Members:

- a. name: string //玩家姓名
- b. stamina: int //耐力值
- c. health: int //健康度
- d. score: int //得分
- e. package: vector< Item > //背包
- f. weaponSum: int //武力值
- g. weight: int //玩家背包中道具的总重量

• Item

- Data Members:

- a. description: string //物品名
- b. longDescription: string //物品详细描述
- c. weightGrams: int //物品重量
- d. value: int //物品价值
- e. weaponCheck: bool //是否为武器类型
- f. weaponValue: int //武器武力值

- g. health: int //健康度加成效果
- h. stamina: int //耐力值加成效果
- i. score: float //得分加成效果

• **Monster**

- Data Members:
 - a. description: string //怪物名称
 - b. health: int //怪物健康度
 - c. weapon: int //怪物武力值
 - d. score: float //怪物分数
 - e. tiredness: bool //怪物疲惫状态 1->疲惫状态 0->正常状态

• **Room**

- Data Members:
 - a. description: string //房间名
 - b. exits: map<string, Room *> //该房间可到达的房间
 - c. itemsInRoom: vector< Item > //房间内所有物品
 - d. isPassed: bool //房间对角色是否可见(是否探索过) 1->可见 0->不可见
 - e. monsterRoomCheck: bool //是否为怪物房间 1->是 0->不是
 - f. monsterList: vector<Monster *> //怪物列表

• **Judge**

- Data Members:
 - a. character: Character * //玩家指针
 - b. monster: Monster * //怪物指针
 - c. preRoom: Room * //角色当前所在房间指针
 - d. nextRoom: Room * //角色想要移动到的房间指针
 - e. moveCheck: bool //角色是否进行移动
 - f. staminaCheck: int //耐力值消耗模式
 - g. winOrLost: int //游戏胜负判定

• **monsterRoomDialog**

- Data Members:
 - a. fightButton: QPushButton* //fight选项按钮
 - b. runButton: QPushButton* //run选项按钮
 - c. label: QLabel*

d. judge: Judge*

- **gameoverDialog**

- Data Members:

- a. fightButton: okButton*

- b. label: QLabel*

- c. Judge* judge

II. 游戏参数描述

1. 角色状态栏

1. 健康度
2. 耐力值
3. 武力值
4. 得分
5. 背包
6. 背包总重量

2. 局内道具

注意：这里道具的生效数值均为基础值，实际生效数值会根据物品的价值进行运算

- 健康度/耐力值分支道具 Health / Stamina

- i. 食物类 Food : Item //继承于Item类

- a. 鼠头鸭脖 duckNeck_actedByMouse

- 质量0 健康度-20 耐力值-20 得分-20

- b. 炒锅饼 fryPancake

- 质量0 健康度+10 耐力值+10 得分+10

- c. 烧烤 barbecue

- 质量0 健康度+20 耐力值+20 得分+20

- d. 异性的好感 like_fromOppositeSex

- 质量0 健康度+30 耐力值+30 得分+10 武力值+10

- e. 一顿冰激凌大餐 bigIceCream

- 质量0 健康度-10 耐力值+30 得分+0 武力值-5

- ii. 学术类 Scholarism : Item

- a. SCI一区论文 sci_firstRegion

- 质量1 健康度+0 耐力值-10 得分+50 武力值+10

b. 学术垃圾 academicGarbage

质量0 健康度+10 耐力值+10 得分-10

c. 导的认可 approve_fromTutor

质量0 健康度+0 耐力值+20 得分+10 武力值+5

d. 导的压迫 pressure_fromTutor

质量2 健康度-10 耐力值-20 得分+10

• 武器类 Weapon : Item

i. 罗技鼠标 logitechMouse

质量1 武力值+20 耐力值+10 得分+10

ii. 计科人抱枕 pillow_CSstudent

质量1 武力值+10 耐力值+20 得分+10

iii. 燃烧的单片机 flamingSinglechip

质量1 武力值+30 耐力值-10 得分+20

iv. 寻迹小车的冲撞 trackingCarDashing

质量2 武力值+30 耐力值-20 得分+30

v. 手感极好的键盘 excellentKeyboard

质量2 武力值+40 耐力值+20 得分+20

3. 怪物

1. 失恋男大 lovelorn_collegeBoy

健康度20 武力值30 分数30

2. 程序的报错、警告和崩溃 bugWarningCrash

健康度40 武力值40 分数50

3. 宿舍小强 cockroachInDormitory

健康度50 武力值20 分数10

4. 局内事件(扩展)

~可以利用自定义弹窗来完成各类事件

III. 游戏流程

游戏背景：

玩家需要扮演一名穿越到地牢世界的大学生，在地牢中生存下来，积极探索地图，获取道具以提高分数，努力达成获胜条件，实现穿越回现实世界的终极目标

在地牢冒险中，大学生勇者会遇到许多与他穿越前相似的道具，这需要他自己去辨别功能，然后更好地利用道具达成终极目标

1. 首先输入角色名字，点击开始游戏选项，等待游戏初始化后开始游戏
2. 每局游戏中，角色会获取固定数值的初始状态，具体如下：
 - i. 健康度: 100
 - ii. 耐力值: 200
 - iii. 武力值: 10
 - iv. 得分: 0
3. 所有地牢房间在初始化后对玩家来说是不可见的，该状态下玩家无法获取房间中怪物和物品信息。只有当玩家进入一个房间后，该房间才会标记为对玩家可见，并且房间中怪物和物品的信息可以被玩家随时查询，以帮助玩家完成探索决策

IV. 游戏机制

0. 数值计算底层机制：

所有涉及到需要变动角色状态值的操作，均按照以下公式处理

$$\text{玩家状态值}+ = (int)(\text{改变量} + 0.5)$$

或

$$\text{玩家状态值}- = (int)(\text{改变量} + 0.5)$$

1. 耐力值消耗

角色每进行一次移动需要耗费基础值10点耐力值，额外的耐力值消耗取决于角色背包中物品的总质量，额外消耗的耐力值与总质量成正比。计算方法如下：

$$\text{每次移动消耗耐力值} = 10 + \text{背包中物品总质量} * 2$$

2. 物品价值

每件 物品/武器 的价值分为[1, 10]十个层次，分别对应[1.1, 2.0]十个档次的额外加成系数。当角色获取道具时触发的道具效果加成与该物品对应的额外加成系数相关联，计算方法如下：

$$\text{实际效果加成} = \text{本身效果加成} * (0.1 * \text{物品价值} + 1.0)$$

3. 胜负条件判定

玩家每次移动后，都会根据相应的状态进行一次游戏胜负的条件判断，若满足其中条件则进行胜负判断并结束游戏，否则继续该局游戏

- 胜利条件

- i. 角色存活，计算此时得分数

ii. 角色存活，得分达到预期阈值

- **失败条件**

i. 在探索完所有地牢房间前，角色健康度降至0

4. 怪物房间

玩家每次进入房间后，若房间内有怪物，则会强制玩家进入战斗环节。每次进入战斗模式均会将该房间标记为可见。该环节有 讨伐 和 逃跑 两种选项：

1. 讨伐 fight()

i. 怪物状态判断

a. **怪物健康度 > 75%**

该状态下，怪物疲惫状态置为0

b. **怪物健康度 ≤ 75%**

该状态下，怪物疲惫状态置为1

角色每次与怪物房间交互后，若怪物疲惫状态为1，则角色在耐力值消耗阶段的最终消耗耐力值*0.8

ii. 玩家与怪物交互

a. **玩家武力值大于怪物武力值：**

该房间内的怪物被打败，怪物健康度降至0，该房间中的怪物被清除掉，同一局内该房间不会再生成怪物；

玩家健康度不变；

玩家可获取 1.0倍的怪物对应积分；

玩家 进入怪物所在房间；

正常消耗耐力值

b. **玩家武力值等于怪物武力值 (优先判定):**

该房间内的怪物被打败 且同一局内不会再生成；

玩家健康度-=怪物武力值*0.1；

触发 斩杀 效果，玩家可获得 1.5倍怪物积分；

玩家 进入怪物所在房间；

正常消耗耐力值

c. **玩家武力值小于怪物武力值：**

怪物不会被打败该房间内怪物健康度-=玩家武力值*0.6；

玩家健康度-=怪物武力值*0.4，玩家不会获取怪物积分；

同时玩家不会进入怪物所在房间，仍 停留在进入怪物房间前的房间内；

耐力值消耗为最终消耗值*0.6

2. 逃跑 run()

无论怪物和玩家武力值大小与否，怪物健康度不改变；

玩家健康度-=怪物武力值*0.4+怪物健康度*0.1；

有道是：“留得青山在，不怕没柴烧。”玩家耐力值会进行一定幅度减少，耐力值消耗为正常情况下最终耐力值消耗*0.4；同时玩家会停留在进入怪物房间前的房间内

5. 角色状态值

道具的增益/减益效果生效后，健康度区间始终为 $[0, 100]$ ，耐力值区间始终为 $[0, 200]$ ，二者数值始终不会超过设定阈值

6. 道具获取

角色每到一个房间后，若为非怪物房间，则会强制将房间中存在的道具加入背包并即时生效；若为怪物房间，则需角色先完成战斗环节，若结果为角色可以进入该怪物房间，则再将房间内所有道具加入背包

7. 初始化

每次开始一局游戏时，将所有道具和怪物随机分配到不同房间中，每个房间最多同时拥有2个道具和1个怪物

所有房间、道具(包括Food, Scholarism, Weapon三个分支类)、角色类型的对象初始实例化过程均放在ZorkUL::init()中，以便统一修改

V. 代码层面执行序

1. 游戏初始化环节

每次开始一局游戏时，首先对游戏中地图进行加载，将道具和怪物房间进行初始化，同时在某一位置进行角色位置初始化

2. 人机交互环节

1. 角色移动阶段

将角色将要移动到的房间存入 *nextRoom 中，判断 *nextRoom 是否指向 NULL，即下一方向是否有房间，若没有则打印撞墙信息并直接结束交互环节

2. 怪物房间判定阶段

输入的房间指针为 *nextRoom，对其进行判断并将其中的Monster对象与Character对象进行交互

- monsterRoomCheck：

0 -> 普通房间

1 -> 怪物房间

i. 怪物房间(两个选项)

a. 讨伐

会先根据怪物健康度改变怪物疲惫状态标志位 tiredness

a. 玩家武力值大于怪物武力值

`moveCheck = 1;`

`staminaCheck = 0;`

b. 玩家武力值等于怪物武力值

`moveCheck = 1;`

`staminaCheck = 0;`

c. 玩家武力值小于怪物武力值

`moveCheck = 0;`

`staminaCheck = 1;`

b. 逃跑

`moveCheck = 0;`

`staminaCheck = 2;`

ii. 非怪物房间

`moveCheck = 1;`

`staminaCheck = 0;`

3. 总判定环节 Judge

每到该环节时，新创建一个 `Judge*` 类型的变量 `*judge`，并将角色指针类型 `Character*`、角色下一步将要到达的房间指针 `*nextRoom` 和当前房间的指针变量 `*currentRoom` 作为参数传入，进行各种状态量的修改和判定。当该环节执行完毕后，随时将该 `Judge*` 类型对象指针销毁并释放空间

i. 耐力值消耗阶段

- 依据 `Character` 内的 `staminaCheck` 进行判定:

0 -> 正常消耗: 1.0倍消耗

1 -> 讨伐情况三: 0.6倍消耗

2 -> 逃跑: 0.4倍消耗

ii. 角色位置判断阶段

- 依据 `Character` 内的 `moveCheck` 进行判定:

0 -> 角色不进行移动

1 -> 角色进行移动

iii. 获取道具阶段

若角色当前所在房间存在道具，则将其全部加入背包并即时生效

iv. 玩家状态值判定阶段

a. 若玩家健康度为0，则将 `winOrLost` 置为0，并直接跳到下一阶段

b. 若玩家耐力值为0，则将 `winOrLost` 置为1

c. 若玩家分数达到设定阈值，则将 `winOrLost` 置为2，并跳到下一阶段

v. 胜负条件判定阶段

- 依据 `ZorkUL` 中的 `winOrLost` 进行判定:

0 -> 游戏失败并结算分数

- 1 -> 游戏胜利并结算分数
- 2 -> 完美胜利并结算分数

3. 本局游戏结束

VI. 问题总结

1. 已解决的问题

- i. 自定义两个类之间互相引用，并在初始化时在A类中实例化B类，出现无法进行实例化的编译错误
- ii. 在头文件中声明了私有指针变量的前提下，在其代码实现时又对同名同类型变量进行new实例化，导致后续使用该变量时无法准确定位该指针变量实际指向的地址，出现段错误(指针乱指)
- iii. 在弹窗中处理传入的指针类型变量，并根据点击的按钮不同实现不同功能

2. 尚未解决的问题

- i. 关于允许移动状态位的更新/判定可能存在bug，只在某些特殊情况下会出现进入房间交互异常的情况
- ii. 胜负判断条件中的完美胜利条件判定存在bug，未在预期步骤完成判定
- iii. 代码结构过于冗杂，功能模块化并未贯彻

VII. 拓展空间

1. 在后续学习中添加数据库技术，实现微型用户登录模式，在单用户上记录分数、游戏局数等数据
2. 加入图形化元素，实现更美观的游戏界面

 Lab4.zip	2024/5/24 20:38	好压 ZIP 压缩文件	5,439 KB
 Lab56 (2).zip	2024/6/7 11:33	好压 ZIP 压缩文件	8,031 KB
 Lab56 (3).zip	2024/6/11 9:40	好压 ZIP 压缩文件	11,308 KB
 Lab56 (4).zip	2024/6/11 21:46	好压 ZIP 压缩文件	11,311 KB
 Lab56 (5).zip	2024/6/12 20:57	好压 ZIP 压缩文件	11,316 KB
 Lab56 (5)_fixedSuccessfully_01.zip	2024/6/11 22:05	好压 ZIP 压缩文件	11,309 KB
 Lab56 (6).zip	2024/6/12 22:38	好压 ZIP 压缩文件	11,325 KB
 Lab56 (7).zip	2024/6/14 12:07	好压 ZIP 压缩文件	11,463 KB
 Lab56 (8).zip	2024/6/17 20:52	好压 ZIP 压缩文件	12,247 KB
 Lab56 (9).zip	2024/6/18 16:47	好压 ZIP 压缩文件	12,269 KB
 Lab56 (9)_monsterInteraction.zip	2024/6/18 9:20	好压 ZIP 压缩文件	12,245 KB
 Lab56 (10).zip	2024/6/18 22:20	好压 ZIP 压缩文件	13,086 KB
 Lab56 (11).zip	2024/6/20 17:00	好压 ZIP 压缩文件	13,079 KB
 Lab56 (12).zip	2024/6/20 18:30	好压 ZIP 压缩文件	13,080 KB
 Lab56_Final_PPT.zip	2024/6/24 19:41	好压 ZIP 压缩文件	88,814 KB
 Lab56(1).zip	2024/6/10 22:06	好压 ZIP 压缩文件	8,011 KB
 Lab56.zip	2024/6/7 21:19	好压 ZIP 压缩文件	8,031 KB
 Lab56_zhou.zip	2024/6/9 21:20	好压 ZIP 压缩文件	8,005 KB
 zhou_lab56_1.zip	2024/6/5 15:31	好压 ZIP 压缩文件	7,493 KB